

DATA STRUCTURES AND ALGORITHM-ECommerceSearchFunction

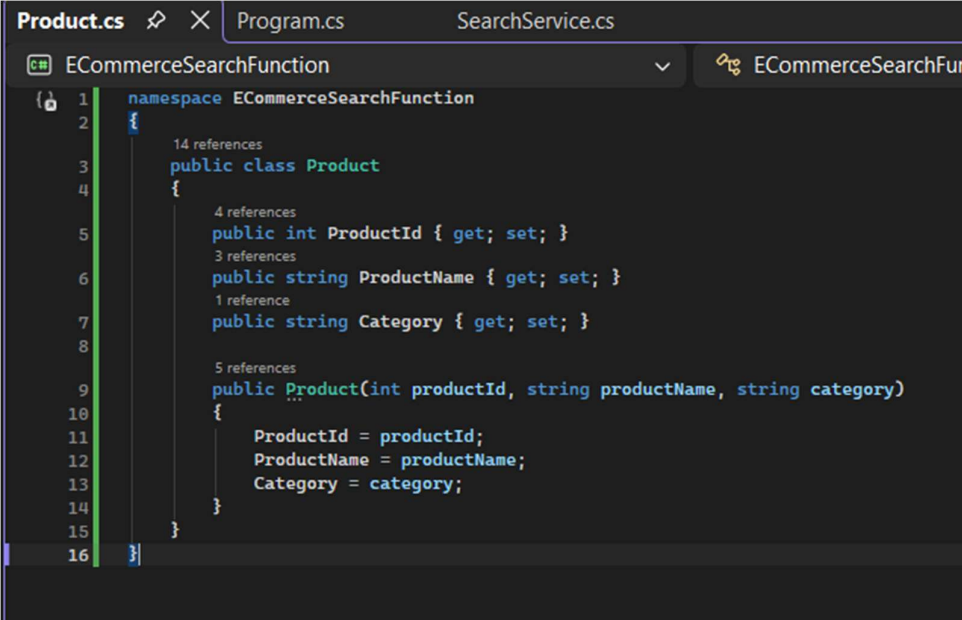
Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:
 - Explain Big O notation and how it helps in analyzing algorithms.
 - Describe the best, average, and worst-case scenarios for search operations.
2. Setup:
 - Create a class Product with attributes for searching, such as productId, productName, and category.
3. Implementation:
 - Implement linear search and binary search algorithms.
 - Store products in an array for linear search and a sorted array for binary search.
4. Analysis:
 - Compare the time complexity of linear and binary search algorithms.
 - Discuss which algorithm is more suitable for your platform and why.

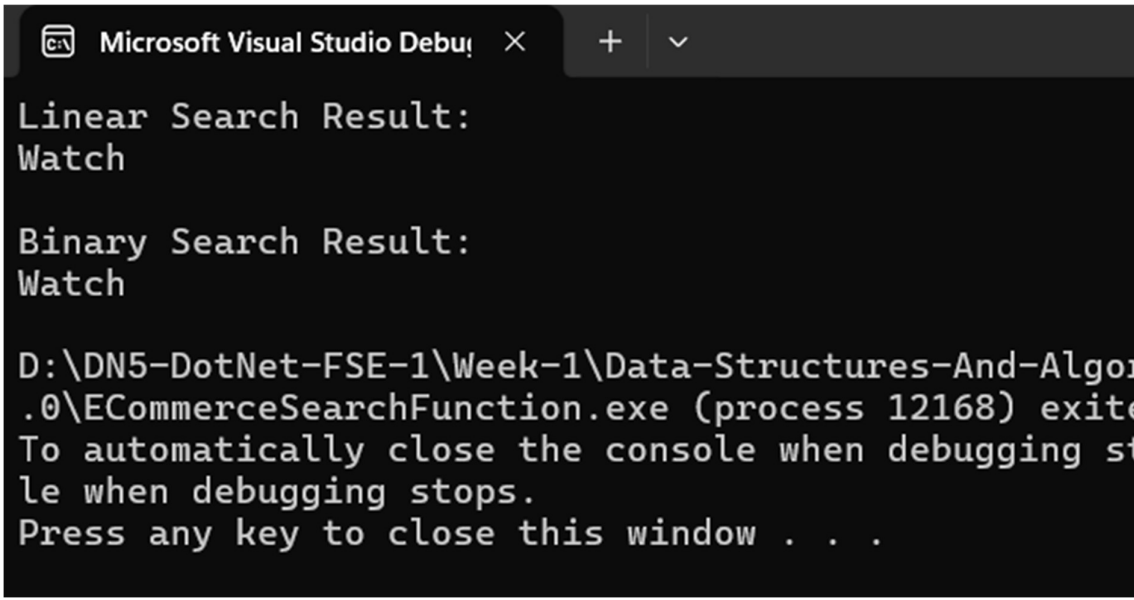


```
Product.cs Program.cs SearchService.cs
ECommerceSearchFunction
1 namespace ECommerceSearchFunction
2 {
3     14 references
4     public class Product
5     {
6         4 references
7         public int ProductId { get; set; }
8         3 references
9         public string ProductName { get; set; }
10        1 reference
11        public string Category { get; set; }
12
13        5 references
14        public Product(int productId, string productName, string category)
15        {
16            ProductId = productId;
17            ProductName = productName;
18            Category = category;
19        }
20    }
21 }
```

```
Product.cs    Program.cs    SearchService.cs
ECommerceSearchFunction
1 namespace ECommerceSearchFunction
2 {
3     2 references
4     public class SearchService
5     {
6         1 reference
7         public static Product LinearSearch(Product[] products, int targetId)
8         {
9             foreach (Product product in products)
10            {
11                if (product.ProductId == targetId)
12                {
13                    return product;
14                }
15            }
16            return null;
17        }
18        1 reference
19        public static Product BinarySearch(Product[] products, int targetId)
20        {
21            int left = 0;
22            int right = products.Length - 1;
23            while (left <= right)
24            {
25                int mid = (left + right) / 2;
26                if (products[mid].ProductId == targetId)
27                {
28                    return products[mid];
29                }
30                if (products[mid].ProductId < targetId)
31                {
32                    left = mid + 1;
33                }
34                else
35                {
36                    right = mid - 1;
37                }
38            }
39            return null;
40        }
41    }
42 }
```

```
Product.cs    Program.cs    SearchService.cs
ECommerceSearchFunction
1 namespace ECommerceSearchFunction
2 {
3     0 references
4     internal class Program
5     {
6         0 references
7         static void Main(string[] args)
8         {
9             Product[] products =
10            {
11                new Product(101, "Laptop", "Electronics"),
12                new Product(102, "Mobile", "Electronics"),
13                new Product(103, "Shoes", "Fashion"),
14                new Product(104, "Watch", "Accessories"),
15                new Product(105, "Bag", "Fashion")
16            };
17            int searchId = 104;
18            Product linearResult =
19                SearchService.LinearSearch(products, searchId);
20            if (linearResult != null)
21            {
22                Console.WriteLine("Linear Search Result:");
23                Console.WriteLine(linearResult.ProductName);
24            }
25            Product binaryResult =
26                SearchService.BinarySearch(products, searchId);
27            if (binaryResult != null)
28            {
29                Console.WriteLine("\nBinary Search Result:");
30                Console.WriteLine(binaryResult.ProductName);
31            }
32        }
33    }
34 }
```

OUTPUT



```
Microsoft Visual Studio Debug Console
Linear Search Result:
Watch

Binary Search Result:
Watch

D:\DN5-DotNet-FSE-1\Week-1\Data-Structures-And-Algorithms\0\ECommerceSearchFunction.exe (process 12168) exited with code 0.
To automatically close the console when debugging stops, enable the setting 'Auto close console when debugging stops'.
Press any key to close this window . . .
```

What is Big O Notation?

Big O notation describes how the running time of an algorithm grows as the input size increases. It helps compare the efficiency of algorithms.

Search Operation Cases

Best Case

- Element found immediately.
- Linear Search: $O(1)$
- Binary Search: $O(1)$

Average Case

- Element found somewhere in the middle.
- Linear Search: $O(n)$
- Binary Search: $O(\log n)$

Worst Case

- Element at the end or not present.
- Linear Search: $O(n)$
- Binary Search: $O(\log n)$

Time Complexity Comparison

Algorithm	Best	Average	Worst
-----------	------	---------	-------

Linear Search	$O(1)$	$O(n)$	$O(n)$
---------------	--------	--------	--------

Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
---------------	--------	-------------	-------------

Which is Better?

Binary Search is more suitable for an e-commerce platform because product data can be maintained in sorted order and searching becomes much faster for large datasets. Linear Search is simple but inefficient for a large number of products.